```
;;; -*- Mode:LISP; -*-

(COMMENT  ;For Maclisp debugging only

(DEFPROP DEFINE-OPERATION (LAMBDA (X) NIL) FEXPR)
(DEFMACRO DEFINE-SYNTAX ((NAME . BVL) . BODY)
  '(DEFMACRO ,NAME ,BVL ,@BODY))
(DEFMACRO LAMBDA (BVL &REST BODY)
  '#'(LAMBDA ,BVL ,@BODY))
(DEFPROP MAP MAPCAR EXPR)
(DEFUN GENERATE-SYMBOL (NAME) (SYMBOLCONC NAME '-17))
(DEFUN NULL? (X) (NULL X))
(DEFVAR ELSE T)
(DEFPROP BEGIN PROGN EXPR)

)

;;; (DEFINE-CONDITION name locals [parents])

(DEFINE-SYNTAX (DEFINE-CONDITION NAME LOCALS . REST)
  (LET ((PARENTS (IF (NULL? REST) '() (CAR REST))))
    '(BEGIN (DEFINE ,NAME (CONDITION ,NAME ,LOCALS ,PARENTS))
            ,@(MAP (LAMBDA (LOCAL)
                     '(DEFINE-OPERATION (,(SYMBOLCONC NAME '- (CAR LOCAL))
                                         ,NAME)))
                   LOCALS))))

(DEFINE-SYNTAX (CONDITION NAME LOCALS PARENTS)
  (LET ((ARGS-VAR (GENERATE-SYMBOL 'ARGS))
        (SELF-VAR (GENERATE-SYMBOL 'SELF)))
    '(OBJECT (LAMBDA (,ARGS-VAR)
               ,(LET ((BODY '(LET ,(MAP (LAMBDA (LOCAL)
                                          '(,(CAR LOCAL)
                                            (COND ((MEMQ ',(CAR LOCAL)
                                                         ,ARGS-VAR)
                                                   => CADR)
                                                  (ELSE ,(CADR LOCAL)))))
                                        LOCALS)
                                  (OBJECT ()
                                    ,@(MAP (LAMBDA (LOCAL)
                                             '((,(SYMBOLCONC NAME '- (CAR LOCAL)))
                                               ,(CAR LOCAL)))
                                           LOCALS)
                                    (=> CONDITION-HANDLER)))))
                  (IF (NULL? PARENTS) BODY
                      '(JOIN ,BODY
                             ,@(MAP (LAMBDA (PARENT)
                                      '(,PARENT ,ARGS-VAR))
                                    PARENTS)))))
             ((IDENTIFICATION SELF) ',NAME)))))

;;; Note:
;;;  Think about: (IF (NOT (GET ... KEY)) (PUT ... KEY default))
;;;  instead. Could save storage, avoid useless computation, simplify
;;;  message passing.

;;; (CONDITION-BIND ((type1 handler1)
;;;                  (type2 handler2)
;;;                  ...)
;;;   . body)
;;;
;;; Handlers must be functions of three args:
;;;   the condition
;;;   a return continuation
;;;   a dismiss continuation

;;; (SIGNAL condition . args)
;;; (ERROR  condition . args)

(DEFINE (SIGNAL CONDITION . ARGS)
  (INVOKE-CONDITION-HANDLERS (CONDITION ARGS) FALSE))
```

```
(DEFINE (ERROR CONDITION . ARGS)
  (INVOKE-CONDITION-HANDLERS (CONDITION ARGS) DEBUGGER))

(DEFINE (INVOKE-CONDITION-HANDLERS CONDITION-OBJECT FAILURE-ACTION)
  (CATCH RETURN
    (DO ((H (FLUID-CONDITION-HANDLER-STATE) (CDR H)))
        ((NULL? H) (FAILURE-ACTION CONDITION-OBJECT))
      (CATCH DECLINE
        (LET ((CLAUSE (CAR H)))
          (IF ((CAR  CLAUSE) CONDITION-OBJECT)
              ((CADR CLAUSE) CONDITION-OBJECT
                             RETURN
                             (LAMBDA () (DECLINE T)))))))))

;;; (DEFINE-PROCEED-TYPE type ...)

;;; (PROCEED-CASE ...)
```

```
;;; Local Modes:
;;; Lisp BIND Indent:1
;;; Lisp CATCH Indent:1
;;; Lisp OBJECT Indent:1
;;; Lisp CONDITION-BIND Indent:1
;;; End:
```

Need to send messages to the USE-VALUE or USE-ANY-VALUE messages
in order to ask what they want to do about prompting. This is analagous
to the :CASE :PROCEED methods of LispM error flavors, but the thing is
that the proceed methods are not a property of the message condition at
all. eg,

```
>>> (DEFINE-PROCEED-CASE (USE-ANY-VALUE CONDITION))

    (DEFINE-PROCEED-CASE (USE-VALUE CONDITION)
      (THE-VALUE (PROMPT-AND-READ NUMBER?
                                  "~%Use what value for ~S instead of ~S: "
                                  (VARIABLE-NAME  CONDITION)
                                  (VARIABLE-VALUE CONDITION))))

    (CONDITION-BIND ((UNBOUND-VARIABLE?
                       (LAMBDA (CONDITION)
                         (PROCEED CONDITION USE-ANY-VALUE)))
                     (UNDEFINED-FUNCTION?
                       (LAMBDA (CONDITION)
                         (PROCEED CONDITION USE-FUNCTION *SOME-FUNCTION*))))
      (EVAL FORM))

==> (DEFINE-OPERATION (USE-ANY-VALUE VALUE-TO-USE)
      ((PROMPT IGNORED-SELF CONDITION G0001)
       (LET* () (G0001))))

    (DEFINE-OPERATION (USE-VALUE VALUE-TO-USE)
      ((PROMPT IGNORED-SELF CONDITION G0001)
       (LET* ((THE-VALUE (PROMPT-AND-READ
                           NUMBER?
                           "~%Use what value for ~S instead of ~S: "
                           (VARIABLE-NAME  CONDITION)
                           (VARIABLE-VALUE CONDITION))))
         (G0001 THE-VALUE))))

    (DEFINE (PROCEED CONDITION PROCEED-TYPE . PROCEED-ARGUMENTS)
      (LET ((CASES (PROCEED-CASES CONDITION)))
        (UNLESS (VALID-PROCEED-CASE? CASES CONDITION)
          (ERROR "~S is not a valid proceed type for ~S"
                 PROCEED-TYPE CONDITION))
        (APPLY PROCEED-TYPE CASES PROCEED-ARGUMENTS)
        (INTERNAL-RETURN-FROM-ERROR)))

    (DEFINE (PROMPTED-PROCEED CONDITION PROCEED-TYPE)
      (PROMPT PROCEED-TYPE CONDITION
              (LAMBDA ARGS
                (APPLY PROCEED CONDITION PROCEED-TYPE ARGS))))
```

This should pretty much describe a non-value-producing error recovery system.

```
>>> (DEFINE (NEGATE X)
      (CHECK-ARG NUMBER? X)
      (- X))

==> (DEFINE (NEGATE X)
      (IF (NOT (NUMBER? X))
          (ERROR WRONG-TYPE-ARGUMENT
                 'VARIABLE-NAME 'X
                 'VARIABLE-VALUE X
                 'PROCEED-CASES
                   (PROCEED-CASE-CLUSTER
                    ((USE-VALUE V) (SETQ X V))
                    ((USE-ANY-VALUE) (SETQ X (RANDOM))))))
      (- X))

==> (DEFINE (NEGATE X)
      (IF (NOT (NUMBER? X))
          (ERROR WRONG-TYPE-ARGUMENT
                 'VARIABLE-NAME 'X
                 'VARIABLE-VALUE X
                 'PROCEED-CASES
                   (OBJECT NIL
                     ((VALID-PROCEED-TYPE? IGNORED-SELF G0001)
                      (OR (EQ? G0001 USE-VALUE)
                          (EQ? G0001 USE-ANY-VALUE)))
                     ((USE-VALUE IGNORED-SELF V G0001)
                      (SETQ X V))
                     ((USE-ANY-VALUE IGNORED-SELF G0001) (SETQ X (RANDOM))))))
      (- X))
```

```
(DEFINE (ERROR TYPE . KEYWORDED-ARGS)
  (LET ((ERROR-OBJECT (APPLY [maker] TYPE KEYWORDED-ARGS)))
    (LET HANDLER-LOOKUP ((HANDLERS (FLUID-HANDLER-STATE)))
         (IF (NULL? HANDLERS)
             (HANDLE-BY-DEFAULT ERROR-OBJECT)
           (LET ((CLAUSE1 (FIRST HANDLERS))
                 (CLAUSES (REST  HANDLERS)))
             (IF ((FIRST CLAUSE1) ERROR-OBJECT)
                 (BIND ;ick
                       ((DISMISS (LAMBDA () (HANDLE-BY-DEFAULT CLAUSES))))
                   (SECOND CLAUSE1))
                 (HANDLE-BY-DEFAULT CLAUSES)))))))
```

(CATCH FOO

(FOO HANDLER-LOOKUP

HANDLER-LOOKUP

Where are all the type predicates coming about?

Where is distinction between simple and complex types?

(DEFINE-SIMPLE-CONDITION

Q: is there some reason can't be:

TRUE → true
FALSE → false
(TRUE) → true
(FALSE) → false
??

```
(DEFINE (NEGATE X)
  (IF (NOT (NUMBER? X))
      (ERROR WRONG-TYPE-ARGUMENT
             'VALUE    X
             'VARIABLE 'X
             'PROCEED-CASES
               (OBJECT ()
                 ((USE-VALUE Y) (SETQ X Y))
                 ((USE-ANY-VALUE $) (SETQ X (RANDOM)))))))
      (- X))
```

OPTIONALITY? Why?
ABILITY TO QUERY WHAT CAN DO.
~~keyword these be optional~~
PRETTY NAMES FOR DEBUGGER

Ah! msg to proceed type can be prompt!

what distinguishes these names from other ops, like print-self, etc.?

Special form? (PROCEED-CASES ...?...)

Q:
Can handler expressions forward?

Can they not forward?

```
(CASES ((USE-VALUE Y) (SETQ X Y))
       ((USE-ANY-VALUE) (SETQ X (RANDOM))))
(OBJECT (LAMBDA (TAG KEY . ARGS)
≡  (CATCH ANY
     (CATCH MAGIC
       (ANY (APPLY KEY ARGS)) (TAG))
  ((USE-VALUE Y)
   (SETQ X Y)
   (MAGIC NIL))
  ...)
```

(DEFINE (SIGNAL ERROR . KEYWORD-ARGS)
  ...

(DEFINE (SIGNAL ERROR KEYS)
  (LET ((OBJ (ERROR)))
    (LET PARSE-KEY ((K KEYS))
      (IF (NULL? K) (OBJ))
        (ELSE ((FIRST K) OBJ (SECOND K))
          (PARSE-KEY (CDDR K)))))
(SETTER ...) (REST (REST ...))

—SIMPLE—

(DEFINE-CONDITION (ERROR. SELF
  #STRING "...")
  (ARGS '()))
(APPLY FORMAT ERROR-OUTPUT STRING ARGS))

(DEFINE-CONDITION UBV
  ((VAR '??))
  ...)
(JOIN UBV ERROR)
(DEFINE-COMPOUND-CONDITION
  ...)

(DEFINE-MACRO (DEFINE-CONDITION NAME VARS PARENTS . DEFAULT)
  (IF (NOT DEFAULT) (SETQ DEFAULT (LIST '(DBG)))
NULL?

(DEBUGGER SELF)

(OBJECT (λ () @DEFAULT)

(PROCEED

Conventions: THE-... eg. THE-FORMAT-STRING
THE-FORMAT-ARGS

Q: ~~can~~ should maker or definer associate fn w/ structure?

(DEFINE-CONDITION (name (self-var) (var1 default1) (var2 default2) ...)
. action)

≡ (DEFINE (name)
(LET ((var1 default1) (var2 default2) ...)

(BEGIN (DEFINE-STRUCTURE-TYPE name var1 var2 ...)
(... default1)
(DEFINE (name . args)
(LET ((self-var (MAKE-name . args)))
(OBJECT (λ () . action)))))

(BEGIN (DEFINE-STRUCTURE-TYPE name (var1 default$_1$) (var$_2$ default$_2$)...)
(DEFINE (name. ARGS)
(LET ((SELF-VAR (APPLY MAKE-NAME ARGS)))
(OBJECT (λ () . action)
⋮
))))

NEEDED:

? Defaults for structure slots
Arg parsing in maker

Local structures.
Possibly,

(WITH-LOCAL-STRUCTURE
---)

Shouldn't structures be potentially funcallable?

What about structures answering extra msgs?

Should there be a general typep msg things can handle — should retvrn pred?

Access structure by kwd?

Alternative to (SET ((STYPE-SELECTOR 'FOO) (STRUCTURE-STYPE X)) 3)
X

must be a path from type name to type. don't like this foo-stype nonsense.

(DEFINE-CONDITION (name self (var$_1$ default$_1$) (var$_2$ default$_2$) ...)
. body)

≡ (WITH-LOCAL-STRUCTURE-TYPE(name ((var$_1$ default$_1$) (var$_2$ default$_2$) ...)

(DEFINE name
(JOIN (OBJECT (λ (). action-body)
⋮)
(MAKE-name))))

Why :Proceed methods ? MMcM

any defaulting
doc association

Should condition-bind-default be searched backward ?

What are restart handlers?

proceed — continue

restart — punt, debug, ...